Greedy Is Not Deterministic: Batch Composition Changes One in Three bf16 LLM Outputs

NeuriCo

Abstract

Practitioners often assume that greedy (temperature-0) decoding makes an LLM's output a deterministic function of its prompt. We test whether this holds when the same prompt is served in different batches. We run 1,700 byte-identical prompts from five tasks through vLLM under static batch sizes $B \in \{1, 4, 16, 64\}$, a same-size reshuffle of batch-mates, and exact repeats. We use ten configurations: five bf16 models (0.5B–3B, QWEN2.5 and LLAMA-3.2) plus fp16, fp32 and AWQ variants. Moving from sequential to batched decoding changes the exact output of 20–34% of prompts for bf16 models, and of 47–76% of long chain-of-thought outputs. This is far above the 3–5% we hypothesized. Batch size itself is not the driver. Reshuffling batch-mates at fixed $B = 64$ causes nearly as much divergence, and repeating an identical schedule causes none in all ten configurations. Every bf16 divergence starts at a near-tie: about 45% at exact top-2 ties and at least 91% within one bf16 rounding step. Higher precision reduces divergence on the same weights from 33.5% (bf16) to 7.3% (fp16) and 2.8% (fp32). Task type has no effect once output length is controlled, and model size shows no trend. Mean accuracy changes by at most 1.8 pp and never significantly after Holm correction, yet 3–5% of items, and up to 16% of chain-of-thought multiple-choice items, flip between correct and incorrect. Single-run bf16 evaluations therefore carry hidden item-level noise, and per-item reproducibility requires a fixed batch schedule, batch-invariant kernels, or higher precision.

1 Introduction

Set the temperature to zero and an LLM should give the same answer to the same prompt every time. Evaluation harnesses, regression tests, answer caches and LLM-as-judge pipelines all rely on this assumption. In practice, however, prompts are almost never decoded alone. Serving engines such as vLLM [Kwon et al., 2023] batch many requests together to use the GPU efficiently, and which requests share a batch depends on traffic, on the evaluation harness, and on configuration choices nobody records.

Does the batch a prompt lands in change its greedy output? Floating-point addition is not associative [Goldberg, 1991]. If a kernel picks a different reduction order or tiling for a different batch shape, the logits of one request can depend on the requests it is co-scheduled with. He and Thinking Machines Lab [2025] argue that this lack of *batch invariance*, rather than run-to-run randomness from atomics, is the main source of LLM inference nondeterminism. Yuan et al. [2025] show that batch size, GPU count and dtype change the accuracy of reasoning models. These works establish that the problem exists. Practitioners still lack a clear picture of its size and shape: how many prompts change, where in the sequence the change begins, which tasks and models are affected, and whether accuracy moves.

What is missing? Most reports vary several factors at once. Batch size changes together with batch composition, task type changes together with output length, and model size changes together with numeric format. Run-to-run noise is seldom measured as a separate floor. As a result, common

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

intuitions remain untested, such as "math diverges more than classification" or "larger models are more stable". The token-level cause of each divergence is also rarely examined.

Our approach. We run a controlled, per-prompt study on open models in a single vLLM engine per model. We submit 1,700 byte-identical prompts in static chunks of $B \in \{1, 4, 16, 64\}$, and add three controls: an exact repeat of the $B = 64$ schedule, a repeat of the $B = 1$ schedule, and a reshuffle that keeps $B = 64$ but changes batch-mates (table 3). The prompts come from five tasks that cross math, knowledge and classification with long and short outputs. In particular, MMLU appears both with chain-of-thought (CoT) and as a single-letter answer, which lets us separate task from length. We record the top-2 log-probabilities of every generated token, so that we can inspect the reference model's margin at the exact token where two runs first differ. We evaluate five bf16 models from 0.5B to 3B parameters in two families, plus fp16, fp32 and AWQ variants of QWEN2.5.

What do we find? Batching changes far more outputs than expected. For bf16 models, 20–34% of all prompts and 47–76% of CoT generations differ between $B = 1$ and $B = 64$ (figure 1). Already at $B = 4$ the effect is at full size, and reshuffling batch-mates at fixed $B = 64$ produces 83–100% of it, while identical schedules agree on all 19,720 repeated generations. The mechanism is simple: about 45% of first-divergence tokens are exact bf16 ties and at least 91% are within one rounding step (0.125 nats), a 40× enrichment over the base rate of ties. Raising precision on fixed weights cuts divergence from 33.5% (bf16) to 7.3% (fp16) and 2.8% (fp32). Mean accuracy stays within ± 1.8 pp, with no significant change after Holm correction, but 3–5% of items flip correctness.

Our contributions are:

- We measure how often greedy outputs change with batching across ten model configurations and 1,700 prompts, and separate the effect of batch size, batch composition and run-to-run noise. Composition, not size, is the variable that matters, and the engine is fully deterministic for a fixed schedule.
- We trace every bf16 divergence to a near-tied top-2 logit and show a monotone precision ladder (bf16 > fp16 > fp32) on identical weights, which identifies rounding of near-ties as the cause.
- We show that the apparent stability of classification is entirely an output-length effect (task type n.s., $p = 0.58$, once log length is controlled), and that model size between 0.5B and 3B has no consistent effect ($\rho = -0.05$).
- We quantify the accuracy impact: mean accuracy is robust, but item-level correctness is not, which matters for regression tests, caching and paired model comparisons.

The rest of the paper is organized as follows. Section 2 reviews related work, section 3 describes our setup, section 4 presents results for each hypothesis, section 5 discusses implications and limitations, and section 6 concludes.

2 Related Work

Nondeterminism in LLM inference. He and Thinking Machines Lab [2025] attribute LLM inference nondeterminism mainly to kernels that are not batch-invariant. Matrix multiplication, RMSNorm and attention choose reduction strategies or tilings based on the batch shape, so the numerics of a request depend on its co-scheduled requests. They also release batch-invariant kernels for vLLM. Yuan et al. [2025] report that GPU count, batch size and dtype change both the greedy outputs and the accuracy of reasoning models, and that fp32 largely restores reproducibility. Atil et al. [2024] document output variation from commercial APIs under nominally deterministic settings. Building on these works, we provide a controlled per-prompt decomposition. We separate batch size from batch composition and from repeat noise, and we locate the exact token and logit margin where each divergence begins.

Floating-point formats in deep learning. Reduced-precision formats are standard for training and inference [Micikevicius et al., 2018]. bfloat16 keeps the fp32 exponent range but has only 8 bits of significand [Kalamkar et al., 2019], so nearby values collapse onto a coarse grid. Weight-only quantization such as AWQ [Lin et al., 2024] stores weights in 4 bits but usually computes activations in fp16. Unlike prior work that compares formats by end-task accuracy, we compare them by how often the argmax flips under batching, and we show that activation precision, not weight quantization, sets the divergence rate.

Table 1: Model configurations. All configurations use the same engine settings. The fp16 and fp32 runs reuse the bf16 weights of QWEN2.5-0.5B, so any difference between them is due to numeric precision alone.

Model	Sizes	Weights / activations
QWEN2.5-Instruct	0.5B, 1.5B, 3B	bf16 / bf16
LLAMA-3.2-Instruct	1B, 3B	bf16 / bf16
QWEN2.5-0.5B-Instruct	0.5B	fp16 / fp16 (cast)
QWEN2.5-0.5B-Instruct	0.5B	fp32 / fp32 (cast)
QWEN2.5-Instruct-AWQ	0.5B, 1.5B, 3B	int4 / fp16

Reproducibility of LLM evaluation. Benchmarks such as GSM8K [Cobbe et al., 2021] and MMLU [Hendrycks et al., 2021] are usually reported from a single greedy run, often with CoT prompting [Wei et al., 2022]. Unlike work that studies variance across prompts or sampling seeds, we isolate a source of variance that remains even under greedy decoding with a fixed prompt. We estimate its size at both the benchmark level and the item level.

3 Method

3.1 Problem formulation

Let x be a prompt and $\mathcal{S}$ a *schedule*, i.e., an assignment of prompts to batches together with the order of batches. Greedy decoding produces tokens $y_t = \arg\max_v \ell_t(v \mid x, y_{<t}; \mathcal{S})$, where ℓ_t are the logits computed by the engine. If the engine were batch-invariant, ℓ_t, and therefore the output $\boldsymbol{y}(x; \mathcal{S})$, would not depend on $\mathcal{S}$. For two schedules $\mathcal{S}_a$ and $\mathcal{S}_b$ we define:

- **Divergence**: $\boldsymbol{y}(x; \mathcal{S}_a) \neq \boldsymbol{y}(x; \mathcal{S}_b)$ as token-id sequences.
- **First divergence index**: the smallest t with $y_t(x; \mathcal{S}_a) \neq y_t(x; \mathcal{S}_b)$.
- **Margin at divergence**: $m = \log p_t^{(1)} - \log p_t^{(2)}$, the gap between the top-1 and top-2 log-probabilities at that index in the reference run $\mathcal{S}_a$ (always $B = 1$ when comparing against the sequential run).
- **Answer change** and **correctness flip**: whether the extracted answer, or its correctness, differs.

3.2 Models

We use ten configurations, all served locally (table 1). We use open models rather than APIs because our question is about the inference engine, which we must control. Five bf16 models span two families and 0.5B–3B parameters: QWEN2.5-{0.5B, 1.5B, 3B}-Instruct [Qwen Team, 2024] and LLAMA-3.2-{1B, 3B}-Instruct [Meta AI, 2024]. To test the floating-point explanation directly, we also run QWEN2.5-0.5B with the same weights cast to fp16 and fp32, and AWQ checkpoints [Lin et al., 2024] of the three QWEN2.5 sizes (int4 weights, fp16 activations).

3.3 Tasks

We build 1,700 items from five tasks (table 2, seed 42). Two tasks produce long CoT outputs and three produce a short label. MMLU appears twice, with disjoint questions: once with CoT and once with a direct letter answer. This lets us compare the same kind of question at two output lengths, so that we can tell the effect of the task apart from the effect of length. All prompts use the model's chat template and a task-specific system prompt. We extract answers with rules ("The answer is N/X" for CoT, label matching for short tasks), and count parse failures as wrong.

3.4 Conditions

We place all prompts in a fixed random order that mixes tasks (order A, seed 0), as a serving queue would see them. For each condition we split this order into consecutive static chunks of size B and call `llm.generate` on each chunk, so that at most B sequences are ever co-scheduled. Table 3

Table 2: Tasks. "Long" tasks ask for step-by-step reasoning that ends in "The answer is . . . ". "Short" tasks ask for a single label.

Task	Source split	n	Output	Max tokens
GSM8K [Cobbe et al., 2021]	test	400	CoT + number	512
MMLU-COT [Hendrycks et al., 2021]	test	300	CoT + letter	512
MMLU-DIRECT [Hendrycks et al., 2021]	test (disjoint)	400	letter	8
AG NEWS [Zhang et al., 2015]	test	300	topic label	8
SST-2 [Socher et al., 2013]	validation	300	positive/negative	8

Table 3: Experimental conditions. All conditions for a model run in the same engine instance. The sequential run `b1` is the reference.

Condition	B	Order	Purpose
`b1`	1	A	sequential reference
`b4`, `b16`, `b64`	4, 16, 64	A	batch-size sweep
`b64_rep`	64	A	identical schedule again (noise floor)
`b64_shuf`	64	B (seed 1)	same size, different batch-mates
`b1_rep`	1	A, every 5th item ($n = 340$)	determinism of the reference

lists the conditions. The two repeat conditions measure the run-to-run noise floor. The reshuffle condition keeps the batch size but changes which prompts share a batch.

Engine settings. We use VLLM 0.32.0 (V1 engine, in-process) with eager execution, chunked prefill (`max_num_batched_tokens=1024`), `max_num_seqs=64`, prefix caching disabled, a maximum model length of 2,048, temperature 0, seed 1234, and top-2 log-probabilities returned for every generated token. We disable prefix caching so that every request computes its own prefill. Shared-GPU time limits forced two reductions for the two 3B bf16 models: `b1` and `b1_rep` ran on the 340-item subset (every 5th item), and `b4` was dropped. `b1_rep` was also skipped for QWEN2.5-3B and QWEN2.5-0.5B-AWQ.

3.5 Statistics

We report Wilson 95% confidence intervals for divergence rates. For accuracy we use paired comparisons: exact McNemar tests [McNemar, 1947] with Holm correction [Holm, 1979] over the 28 primary tests (model $\times$ $B \in \{4, 16, 64\}$ against $B = 1$), and 2,000-sample paired bootstrap 95% CIs on the accuracy difference. To separate length from task, we fit a logistic regression of divergence on $\log(\text{output length})$, task type and model. We also compute a per-token hazard, the number of divergences per 1,000 identical tokens at risk, which normalizes for length. For model size we use Spearman's ρ over the five bf16 models.

Hardware. We run on 4$\times$ RTX A6000 (48 GB, Ampere) GPUs shared with other users, with each job confined to a 3.4–8.8 GB memory slice. Software: Python 3.12, VLLM 0.32.0, PyTorch 2.13 (CUDA 13.0), transformers 5.19. Generation took about 19 GPU-hours in total (about 10.5k generations per configuration).

4 Results

We organize the results around five hypotheses. H1: at least 95% of prompts give identical outputs across batch sizes. H2: divergence grows with output length. H3: math and reasoning diverge more than classification. H4: mean accuracy changes by less than 1 pp while individual items flip. H5: smaller models diverge more. Table 8 in section 5 summarizes the verdicts.

4.1 How often do greedy outputs change?

H1 is refuted for bf16. Pooled over all 1,700 prompts, the share of outputs that change between $B = 1$ and $B = 64$ is 33.5% [31.3, 35.7] for QWEN2.5-0.5B, 19.9% for QWEN2.5-1.5B, 33.2% for QWEN2.5-3B ($n = 340$), 23.7% for LLAMA-3.2-1B and 23.2% for LLAMA-3.2-3B ($n = 340$).

Table 4: Percentage of prompts whose greedy output is *not* token-identical between two conditions, shown as long (CoT, $n = 700$) / short (label, $n = 1000$). For the 3B bf16 models, comparisons against `b1` use the 140/200-item subsets, and `b4` was not run (–). Repeats are exactly zero everywhere. Reshuffling batch-mates at fixed $B = 64$ gives almost the same divergence as going from $B = 1$ to $B = 64$. Lowest (most stable) values per column in **bold**.

		Batch size vs. $B = 1$			Controls		
Model	**dtype**	`b1 vs b4`	`b1 vs b16`	`b1 vs b64`	`b64 vs b64_shuf`	`b64 vs b64_rep`	`b1 vs b1_rep`
QWEN2.5-0.5B	bf16	75.9 / 3.6	75.0 / 3.7	76.4 / 3.4	72.7 / 1.3	0 / 0	0 / 0
QWEN2.5-1.5B	bf16	45.1 / 1.9	44.0 / 1.1	46.9 / 1.1	40.0 / 0.3	0 / 0	0 / 0
QWEN2.5-3B	bf16	–	65.7 / 2.5	76.4 / 3.0	63.7 / **0.0**	0 / 0	–
LLAMA-3.2-1B	bf16	50.1 / 2.3	50.1 / 4.3	51.7 / 4.1	45.3 / 0.4	0 / 0	0 / 0
LLAMA-3.2-3B	bf16	–	57.1 / 1.0	54.3 / 1.5	47.0 / 0.4	0 / 0	0 / 0
QWEN2.5-0.5B	fp16	17.9 / 0.5	20.7 / 0.4	17.3 / 0.3	16.1 / **0.0**	0 / 0	0 / 0
QWEN2.5-0.5B	fp32	**6.1** / 0.3	7.4 / 0.3	**6.3** / 0.4	**4.9** / **0.0**	0 / 0	0 / 0
QWEN2.5-0.5B	AWQ	18.4 / 0.4	18.0 / 0.3	20.1 / 0.2	19.9 / 0.4	0 / 0	–
QWEN2.5-1.5B	AWQ	7.9 / 0.2	**6.0** / 0.1	7.4 / 0.3	8.7 / 0.1	0 / 0	0 / 0
QWEN2.5-3B	AWQ	15.0 / **0.0**	14.4 / **0.0**	13.6 / **0.0**	13.9 / **0.0**	0 / 0	0 / 0

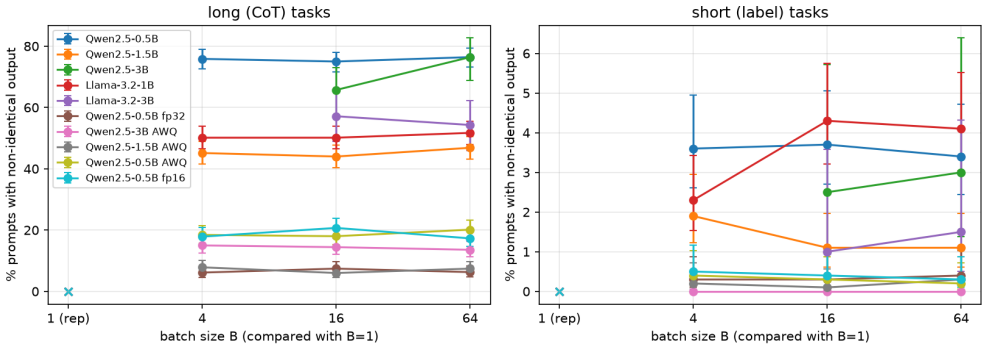


Figure 1: Share of prompts whose output differs from the sequential ($B = 1$) run, for long CoT tasks *(Left)*and short label tasks *(Right)*. Error bars are Wilson 95% CIs; "1 (rep)" is the $B = 1$ repeat, which never diverges. bf16 models (top five lines in the left panel) diverge on 45–76% of CoT prompts already at $B = 4$, and the rate does not grow with B. fp16, fp32 and AWQ configurations sit far lower.

This is 4–10× the 3–5% we hypothesized. On long CoT tasks the rate is 47–76%, and on short label tasks it is 1–4% (table 4). The *extracted answer* changes for 5–12.5% of prompts in bf16. H1 holds only outside bf16: fp32 changes 2.8% [2.1, 3.7] of prompts, fp16 7.3%, and AWQ 3.2–8.4%.

There is no dose-response in batch size. Figure 1 shows that $B = 4$ already produces the full effect and $B = 64$ adds nothing. What matters is *whether* the numerics of a request change, not by how much the batch grows.

Batch composition matters as much as batch size. Keeping $B = 64$ and only changing which prompts share a batch (`b64` vs `b64_shuf`) reproduces 83–100% of the $B = 1$ vs $B = 64$ divergence on CoT tasks (e.g., 72.7% vs 76.4% for QWEN2.5-0.5B). In a serving system, batch composition depends on traffic, so even a fixed batch-size setting does not give reproducible outputs.

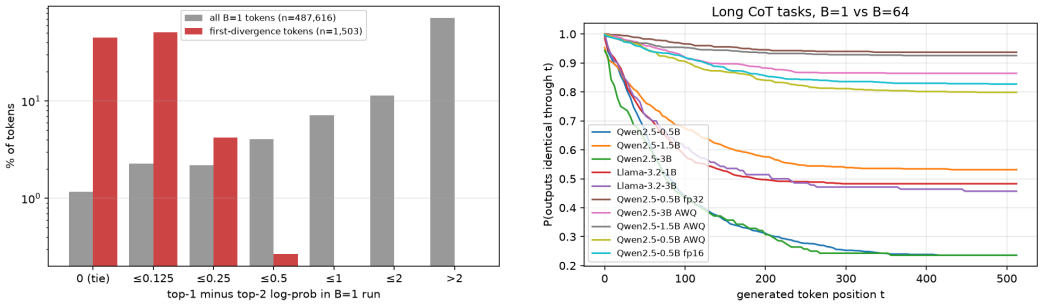
The engine is deterministic for a fixed schedule. All 18 repeat comparisons (10 `b64_rep` and 8 `b1_rep`) agree on every prompt: 0 divergences out of 19,720 generations. The variation we observe is therefore deterministic *batch dependence*, not run-to-run randomness.

4.2 Where do divergences start?

Every bf16 divergence starts at a near-tie. bf16 stores logits with an 8-bit significand, so log-probabilities near the top of the distribution fall on a grid with steps of 0.0625–0.125 nats. Only about 1% of all generated tokens are exact top-2 ties. Yet 43–47% of first-divergence tokens are

Table 5: Logit margin at the first divergence token (`b1` vs `b64`). "Tie" means the top-1 and top-2 log-probabilities in the `b1` run are exactly equal at the model's precision. Ties make up only 1–1.5% of all bf16 tokens but 43–47% of first-divergence tokens. Margins are in nats.

Model (dtype)	**% all tokens tied**	**% divergences tied**	**% divergences $m \leq 0.125$**	**Median m**
QWEN2.5-0.5B (bf16)	1.02	43.2	93.9	0.125
QWEN2.5-1.5B (bf16)	0.93	46.9	96.2	0.125
QWEN2.5-3B (bf16)	1.38	43.4	91.2	0.125
LLAMA-3.2-1B (bf16)	1.55	45.2	97.8	0.125
LLAMA-3.2-3B (bf16)	1.52	46.8	100	0.125
QWEN2.5-0.5B (fp16)	0.14	46.8	100	0.016
QWEN2.5-0.5B (fp32)	0.00	0	100	0.0014
QWEN2.5-AWQ (0.5/1.5/3B)	0.12–0.14	45–60	100	0.000–0.016



(a) Margin distribution (bf16, log scale)

(b) Identical-prefix survival (CoT tasks)

Figure 2: *(a)*Top-1 minus top-2 log-probability in the $B = 1$ run for all generated tokens (gray, $n = 487{,}616$) and for first-divergence tokens (red, $n = 1{,}503$). Divergences sit almost entirely at exact ties or within one bf16 step (0.125 nats), and never at margins above 0.5. *(b)*Probability that the $B = 1$ and $B = 64$ outputs are still identical after t generated tokens. bf16 curves drop steeply in the first 100–200 tokens and level off at 24–53%; fp32 stays above 93%.

exact ties, a roughly 40× enrichment, and 91–100% lie within one rounding step (table 5, figure 2a). The largest margin at any divergence was 0.375 nats for bf16 and 0.047 nats for fp16 and fp32. Batch-dependent reduction order perturbs the logits by about one rounding step, which is enough to flip the argmax only where two candidates are (nearly) tied.

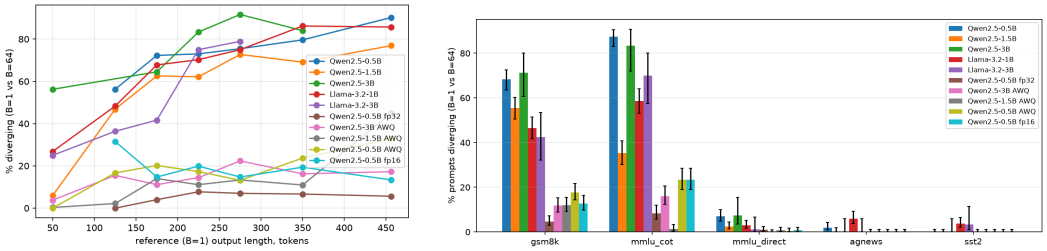
Precision controls the rate. On identical QWEN2.5-0.5B weights, the share of prompts that diverge between $B = 1$ and $B = 64$ falls monotonically from 33.5% (bf16) to 7.3% (fp16) and 2.8% (fp32). Finer formats produce fewer ties (0.14% of tokens in fp16, essentially none in fp32), and their divergences occur at much smaller margins (median 0.016 and 0.0014 nats). The AWQ models store 4-bit weights but compute in fp16, and they diverge at the fp16 level (3–8%), well below their bf16 counterparts. So activation precision, not weight quantization, sets the rate.

Divergences do not re-synchronize. Once two runs choose different tokens at a near-tie, their continuations differ for the rest of the output (figure 2b). For bf16, the median first divergence in CoT outputs is 45–57 tokens in, at 20–40% of the output length (60–110 tokens for fp16, fp32 and AWQ). For bf16 short-label tasks, 44–100% of divergences occur at the very first generated token, which is decided by the prefill pass.

4.3 Length versus task type

H2 is supported: longer outputs diverge more. Each generated token is another chance to hit a near-tie, so divergence rises steeply with output length (figure 3a). In a logistic regression on CoT items ($B = 1$ vs $B = 64$, all models), the coefficient on log length is $+1.14$ ($p \approx 10^{-78}$). Across all tasks it is $+0.95$ ($p \approx 10^{-67}$).

H3 is refuted as stated: the task effect is a length effect. Classification looks far more stable than CoT (1–4% vs 47–76%, figure 3b), but its outputs are only 1–3 tokens long. Once log length is in



(a) Divergence vs. output length (CoT)

(b) Divergence by task

Figure 3: *(a)*Share of CoT prompts that diverge ($B = 1$ vs $B = 64$), binned by the length of the $B = 1$ output (bins with $\geq$10 items). bf16 divergence rises from 6–56% for $\sim$50-token outputs to 77–90% beyond 400 tokens. *(b)*Divergence by task with Wilson 95% CIs. Label tasks look stable, but their outputs are only 1–3 tokens long; per token they are not more stable (section 4.3).

Table 6: Accuracy under sequential (`b1`) and batched (`b64`) decoding. Δ is `b64` minus `b1` with a paired bootstrap 95% CI. Flips count items that go right$\rightarrow$wrong (R$\rightarrow$W) or wrong$\rightarrow$right (W$\rightarrow$R). p is the exact McNemar test, with the Holm-adjusted value over 28 tests in parentheses. No difference is significant, but bf16 models flip 3–5% of items. 3B bf16 models use the 340-item subset. Lowest flip rate in **bold**.

Model	**dtype**	**Acc** `b1`	**Acc** `b64`	Δ (pp) [95% CI]	**R$\rightarrow$W**	**W$\rightarrow$R**	**Flip %**	p (Holm)
QWEN2.5-0.5B	bf16	56.18	56.88	+0.71 [−0.41, +1.76]	39	51	5.3	0.25 (1.0)
QWEN2.5-1.5B	bf16	70.88	70.59	−0.29 [−1.06, +0.59]	29	24	3.1	0.58 (1.0)
QWEN2.5-3B	bf16	72.94	71.76	−1.18 [−2.94, +0.59]	7	3	2.9	0.34 (1.0)
LLAMA-3.2-1B	bf16	50.12	49.94	−0.18 [−1.12, +0.76]	35	32	3.9	0.81 (1.0)
LLAMA-3.2-3B	bf16	67.65	69.41	+1.76 [−0.29, +4.12]	4	10	4.1	0.18 (1.0)
QWEN2.5-0.5B	fp16	56.82	56.82	0.00 [−0.41, +0.47]	7	7	0.8	1.0 (1.0)
QWEN2.5-0.5B	fp32	56.35	56.12	−0.24 [−0.65, +0.18]	8	4	0.7	0.39 (1.0)
QWEN2.5-0.5B	AWQ	45.00	45.24	+0.24 [−0.18, +0.65]	4	8	0.7	0.39 (1.0)
QWEN2.5-1.5B	AWQ	67.47	67.35	−0.12 [−0.47, +0.18]	5	3	0.5	0.73 (1.0)
QWEN2.5-3B	AWQ	74.53	74.59	+0.06 [−0.18, +0.29]	2	3	**0.3**	1.0 (1.0)

the regression, the coefficient for short-label vs CoT is -0.14 with $p = 0.58$. The per-token hazard (divergences per 1,000 tokens at risk) is *not* lower for label tasks. For bf16 models it is 3.7–5.8 for GSM8K, 5.7–12.9 for MMLU-COT, 5.8–36.6 for MMLU-DIRECT, and 0–31 for AG NEWS and SST-2. The first token of an uncertain multiple-choice answer is especially prone to ties. Within CoT tasks, math diverges *less* than knowledge questions: at equal length, GSM8K has an odds ratio of 0.41 relative to MMLU-COT ($p \approx 5 \times 10^{-31}$). One plausible reading is that arithmetic steps have more peaked next-token distributions, and so fewer near-ties, than free-text explanations of answer options.

4.4 Does accuracy change?

H4 is supported: mean accuracy is robust, item correctness is not. Across the five bf16 models, the mean absolute accuracy change between $B = 1$ and $B = 64$ is 0.8 pp, and no single change exceeds 1.8 pp (table 6, figure 4). None of the 28 primary tests is significant after Holm correction; the smallest raw p is 0.002 (fp16, $B = 1$ vs $B = 4$, -0.76 pp), which becomes 0.066 after adjustment. Flips are symmetric (R$\rightarrow$W $\approx$ W$\rightarrow$R), so batching neither helps nor hurts on average. It re-rolls borderline items. Still, 2.9–5.3% of all items flip correctness for bf16 models, and the rate is much higher on CoT multiple-choice: 16.3% of MMLU-COT items flip for QWEN2.5-0.5B, 11.7% for LLAMA-3.2-3B, 8.3% for QWEN2.5-3B and 7.7% for LLAMA-3.2-1B. One exploratory comparison outside the pre-specified family is nominally significant (QWEN2.5-3B, `b64` vs `b64_shuf`, +0.94 pp, $p = 0.014$ uncorrected). With 10 such shuffle tests, this is consistent with chance.

Example. On a GSM8K item, QWEN2.5-1.5B at $B = 1$ writes "Initially, Hunter counted 50 cars." while at $B = 64$ it writes "…50 cars *in the parking lot*." The runs split at token 19, where the $B = 1$

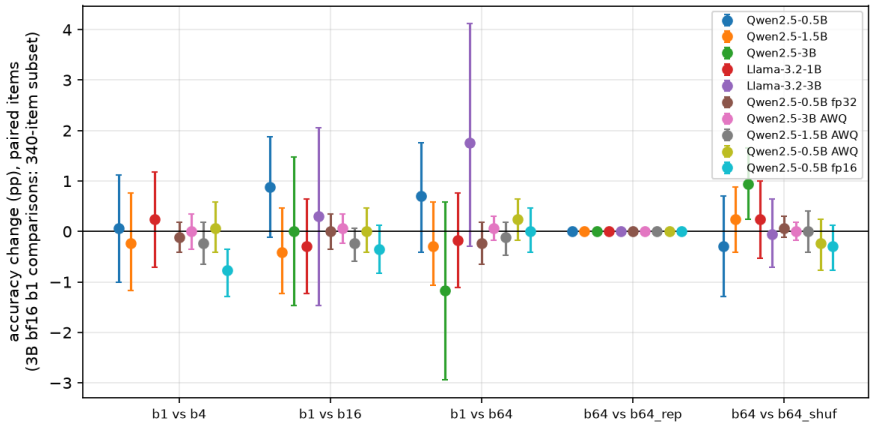


Figure 4: Paired accuracy change (percentage points) with bootstrap 95% CIs for each comparison and model. Almost all intervals include zero, and no difference survives Holm correction; the identical-schedule repeat (`b64` vs `b64_rep`) is exactly zero. Changes reach $\pm$1–1.8 pp in single comparisons.

Table 7: CoT divergence (%) between `b64` and `b64_shuf` ($n = 700$) by model size. There is no monotone trend; QWEN2.5-1.5B is the most stable size in both dtypes.

	0.5B / 1B	**1.5B**	**3B**
QWEN2.5 (bf16)	72.7	**40.0**	63.7
QWEN2.5 (AWQ)	19.9	**8.7**	13.9
LLAMA-3.2 (bf16)	**45.3** (1B)	–	47.0

margin is exactly one bf16 step (0.125). The $B = 1$ run then reasons to the correct answer (35), while the $B = 64$ run adds an extra, wrong step and answers 55. More examples are in appendix A.

4.5 Does model size matter?

H5 is not supported. We find no monotone relation between model size and divergence (Spearman $\rho = -0.05$, $p = 0.93$ over the five bf16 models, CoT, $B = 1$ vs $B = 64$). QWEN2.5-1.5B is the most stable QWEN2.5 model in both bf16 and AWQ, and QWEN2.5-3B diverges about as often as 0.5B (table 7). Size effects are small and model-specific, while dtype changes the rate by 5–15$\times$. Larger models also write different and often longer CoTs, which could confound the comparison, but the length bins in figure 3a show no consistent size ordering either.

5 Discussion

What does this mean for evaluation? A bf16 benchmark score is one sample from a distribution over batch schedules. In our runs the spread is about $\pm$1 pp for mean accuracy and up to 1.8 pp for a single comparison. That is the same size as many reported gains between methods or checkpoints. Two evaluations that differ only in harness batch size, server load or request order can disagree on 3–5% of items, and on up to 16% of CoT multiple-choice items. This matters most where people compare items rather than averages: regression tests that check outputs exactly, caches keyed on the prompt, audits that replay a decision, and paired model comparisons. Before interpreting item-level differences between two configurations, one should measure a same-configuration baseline. Our `b64_shuf` control shows how large this null difference can be.

Why does the effect not scale with B? The near-tie mechanism explains the absence of a dose-response. A batch change does not add error that grows with B. It changes the reduction order, which moves logits by about one rounding step. Any such change flips the argmax at the same set

Table 8: Summary of hypothesis verdicts.

ID	Claim	Verdict
H1	$\geq$95% identical; only 3–5% diverge	**Refuted** for bf16 (66–80% identical overall, 24–53% on CoT); holds for fp32 (97%) and roughly for fp16/AWQ (92–97%)
H1b	Cause is floating-point effects in batched kernels	**Supported**: deterministic batch-dependent rounding acting at near-tied logits
H2	Divergence grows with length	**Supported** (log-length coefficient $+1.14$, $p \approx 10^{-78}$)
H3	Math/reasoning diverges more than classification	**Refuted as stated**: task n.s. given length ($p = 0.58$); GSM8K diverges *less* than MMLU-CoT (OR 0.41)
H4	Mean accuracy $<$1 pp; items flip	**Supported**: mean $\lvert\Delta\rvert$ = 0.8 pp, none significant; 2.9–5.3% of items flip
H5	Smaller models diverge more	**Not supported** ($\rho = -0.05$, $p = 0.93$)

of tied positions, whether B goes from 1 to 4 or to 64. This is also why reshuffling batch-mates is almost as disruptive as changing B, and why identical schedules agree exactly.

Why does length dominate task? Per token, the chance of a flip is a few per thousand, set by how often the model sees a near-tie. Over hundreds of CoT tokens these chances add up, while a one-token label has only one chance. Short multiple-choice answers in fact have a *higher* per-token hazard, because the model is often unsure between two letters. Claims that "reasoning is fragile" under batching are therefore mostly claims about long outputs.

What can practitioners do? Three options follow from our results. (1) Fix the batch schedule and report it, accepting that results are reproducible only under that schedule. (2) Use batch-invariant kernels [He and Thinking Machines Lab, 2025], which remove the dependence at a throughput cost. (3) Raise precision: on identical weights, fp32 cuts divergence about 12$\times$ and fp16 about 5$\times$ relative to bf16. fp16 is a cheap partial fix where its range suffices.

5.1 Limitations

Scale. Our largest model has 3B parameters. A planned QWEN2.5-7B bf16 run never obtained the required $\geq$19 GB slice on the shared GPUs. The tie mechanism does not depend on model size, but we have not measured rates for models of 7B and above.

One engine, one GPU type. All runs use vLLM 0.32 on Ampere GPUs in eager mode with chunked prefill at 1,024 tokens. CUDA graphs, other attention backends, tensor parallelism, other engines or Hopper GPUs may differ in invariance. We expect exact rates to be engine-specific, while the qualitative findings (near-tie mechanism, precision ladder, length dependence) should transfer.

Static batches. We control batch size with static chunks. Production continuous batching mixes prefill and decode of requests arriving at random times, which probably makes composition even more variable than in our setting.

Subsets and model strength. The two 3B bf16 models ran $B = 1$ on 340 items and skipped $B = 4$, so their intervals are wider. Base accuracies are 45–76%, so many items are borderline; a strong model on easy items would likely flip fewer.

Measurement. Answer extraction is rule-based, and parse failures count as wrong (up to 8% of MMLU-CoT outputs for QWEN2.5-0.5B). Because comparisons are paired, this affects both conditions equally. Margins come from the log-probabilities returned by vLLM, so "exact tie" means equal at that precision.

Broader impact. This work concerns the reliability of LLM evaluation and serving. Its main risk is misreading: a non-significant mean change could be read as "batching is harmless," while item-level behavior, which matters for audits and user-facing decisions, is not stable. We report both levels to avoid this.

6 Conclusion

We asked whether the batch a prompt is processed in changes its greedy output. In a standard bf16 vLLM deployment, it does: changing batch size or batch-mates changes the exact output of 20–34% of prompts and 47–76% of CoT generations, while identical schedules reproduce outputs exactly. Every divergence begins at a near-tied logit, where batch-dependent rounding flips the argmax, and the continuation then drifts freely. Output length, not task type, determines how exposed a prompt is, and numeric precision matters far more than model size between 0.5B and 3B. Mean accuracy is robust to batching, but 3–5% of items, and up to 16% of CoT multiple-choice items, flip correctness.

The key takeaway is that *greedy decoding in bf16 is deterministic only given the batch schedule*. Evaluations should treat a single run as one sample, and item-level reproducibility requires a fixed schedule, batch-invariant kernels, or higher precision.

Future work should test models of 7B–70B parameters and Hopper GPUs, use vLLM's batch-invariant mode as a positive control, test whether computing only the final projection (`lm_head`) in fp32 removes most ties, simulate continuous batching under realistic arrival processes, and study reasoning models with 4k–32k-token traces, where our length results predict near-certain divergence.

References

Berk Atil et al. LLM stability: A detailed analysis with some surprises. *arXiv preprint arXiv:2408.04667*, 2024.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

David Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, 1991.

Horace He and Thinking Machines Lab. Defeating nondeterminism in LLM inference. Thinking Machines Lab: Connectionism, 2025. Blog post.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

Dhiraj Kalamkar et al. A study of BFLOAT16 for deep learning training. *arXiv preprint arXiv:1905.12322*, 2019.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.

Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

Meta AI. Llama 3.2: Revolutionizing edge AI and vision with open, customizable models. Meta AI blog, 2024.

Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations (ICLR)*, 2018.

Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of EMNLP*, 2013.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Jiayi Yuan et al. Give me FP32 or give me death? challenges and solutions for reproducible reasoning. *arXiv preprint arXiv:2506.09501*, 2025.

Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

A Divergence Examples

We show further $B = 1$ vs $B = 64$ pairs. In each case the first differing token has a $B = 1$ margin of 0.125 nats (one bf16 step) or 0 (exact tie). Outputs are truncated with "…".

LLAMA-3.2-1B, GSM8K (gold 9; margin 0.125 at token 26). Both runs start "To find the minimum number of points Jane must score on the third test, we need to first find the total points she needs to…". The $B = 1$ run continues "…score on all three tests to pass", correctly computes $42 - 33 = 9$, but then adds 18 and answers 27 (wrong). The $B = 64$ run continues "…pass", sets up $15 + 18 + x = 42$ and answers 9 (correct).

LLAMA-3.2-1B, AG NEWS (gold World; margin 0 at token 0). The $B = 1$ run outputs "World" and the $B = 64$ run outputs "Business". The two labels are exactly tied in bf16 at the first token, so the prefill pass alone decides the label.

QWEN2.5-3B, MMLU-COT (gold C; margin 0 at token 6). Both runs begin "The woman's purported acceptance was…". The $B = 1$ run continues "in the form of a fax", reasons for 177 tokens and answers C (correct). The $B = 64$ run continues "oral", concludes after 42 tokens that the statute of frauds applies, and answers B (wrong).

QWEN2.5-3B, GSM8K (gold 2; margin 0 at token 49). Both runs compute 8 test tubes. At token 49 the $B = 1$ run writes "Next, we know Igor gathered 2 more beakers" and the $B = 64$ run writes "Next, let's find out how many Petri dishes". The $B = 1$ run goes on to solve for the wrong quantity, while the $B = 64$ run reaches the correct answer, 2.

B Reproducibility Details

Prompts are stored in `datasets/processed/items.jsonl`. Raw generations, including token ids and per-token top-2 log-probabilities, are stored per model and condition in `results/gen/<model>/<cond>.jsonl`, with timings in `timing.jsonl`. Order A uses seed 0, order B (for `b64_shuf`) uses seed 1, the item sample uses seed 42, and the engine seed is 1234. The 340-item subset consists of every 5th item in order A. The Holm family contains the 28 tests of model $\times$ $B \in \{4, 16, 64\}$ against $B = 1$ (24 for the eight configurations with `b4`, plus 4 for the two 3B bf16 models without it). The per-token hazard divides the number of diverging items by the number of tokens at risk, summed over items: for a diverging pair, the tokens up to and including the first divergence; for an identical pair, the full output length.